

Agentic Natural Language Querying Across Structured Databases

Shreyas Mohite
San Jose State University
San Jose, CA

Shubham Naik
San Jose State University
San Jose, CA

Rutuja Kadam
San Jose State University
San Jose, CA

Abstract—Traditional single-agent Text-to-SQL systems frequently suffer from fragility in complex query planning, a lack of error recovery mechanisms, and critical security vulnerabilities regarding destructive commands. This study offers a strong, multi-agent architecture that is intended to get around these restrictions by tightly enforcing safety and improving reliability. Using the LangGraph framework, we created a stateful agentic workflow by breaking down the generation process into four specialized agents: a Controller for error validation and self-correction, an Executor for read-only operations, a Planner for logical decomposition, and a Synthesizer for code generation. We trained the Qwen2.5-Coder-7B-Instruct model on the Spider dataset supplemented with 2,000 synthetic adversarial cases to explicitly reject malicious inputs like DROP or DELETE in order to implement a novel “Defensive SQLite Expert” fine-tuning method. A 98% Safety Detection Rate is shown by experimental findings on the Chinook database, where 100% of damaging queries are blocked while there are no false positives. The solution successfully balanced high utility with enterprise-grade security, achieving an overall success rate of 80.9%.

Index Terms—Text-to-SQL, Multi-Agent Systems, LLM Safety, Database Security, LangGraph

I. INTRODUCTION

A. Problem Definition

Recent developments in Large Language Models (LLMs) have greatly expedited the democratization of access to structured data via Natural Language Interfaces (NLIs). However, the intrinsic drawbacks of conventional, monolithic Text-to-SQL infrastructures continue to hinder the use of these models in enterprise settings. When faced with the complexity of real-world database schemas, single-agent systems show serious flaws even though they are capable of zero-shot generalization. These restrictions consist of three parts:

- **Stochastic Fragility:** When the query complexity surpasses the model’s immediate context window, monolithic models sometimes fail to decompose complex semantic demands, leading to “hallucinated” schema references and logically unsound SQL production.
- **Absence of Error Recovery:** There is no feedback loop in the widely used “single-pass” generating paradigm. The system cannot read the database engine’s error trace to iteratively improve the syntax when a generated query causes an execution error, which results in a catastrophic interaction failure.
- **Safety & Alignment Deficits:** Most importantly, database security protocols are not inherently aligned

with general-purpose LLMs. These models are still vulnerable to prompt injection attacks in the absence of thorough fine-tuning on adversarial vectors. They can easily produce destructive instructions (such as DROP, DELETE, TRUNCATE) that jeopardize the referential integrity and persistence of the underlying data.

B. Motivation

The conflict between the probabilistic capabilities of existing LLMs and the deterministic security needs of enterprise data systems is the driving force behind this study. There is a dearth of research on “defensive generation,” or a model’s ability to consistently identify and reject harmful intent, despite the fact that much recent literature has optimized for generation accuracy on common benchmarks (such as Spider). Text-to-SQL systems need to go beyond simple translation accuracy in order to move from academic prototypes to practical production deliverables. They need an architecture that uses adversarial hardening to ensure safety and semantic grounding to ensure operational resilience. According to this research, the only way to attain such reliability in a multi-agent system is to separate generation from reasoning.

C. Objectives

This research attempts to design and evaluate a safe, multi-agent Text-to-SQL system in order to address these architectural and safety flaws. The particular goals of the study are:

- **Architectural Decomposition:** Using the LangGraph framework, create a stateful, agentic workflow that divides the cognitive load into specialized roles: a *Executor* for runtime validation, a *Synthesizer* for code creation, and a *Planner* for logical decomposition.
- **Semantic Grounding via RAG:** Using dynamic Schema Cards and a domain-specific Business Glossary, a Retrieval-Augmented Generation (RAG) technique is implemented to reduce hallucinatory errors by anchoring the agents on a verifiable ground truth.
- **Adversarial Hardening:** In order to achieve a 100% rejection rate for destructive queries and enforce a rigorous, non-negotiable policy of read-only access, the Qwen2.5-Coder-7B-Instruct model will be refined on a dataset supplemented with 2,000 synthetic adversarial cases in order to create a “Defensive SQLite Expert.”

II. RELATED WORK

A. Monolithic Text-to-SQL Architectures

Monolithic, single-pass architectures have long dominated the field of text-to-sql generation. Sequence-to-sequence (Seq2Seq) models or rule-based templates were the foundation of early methods. With models like RAT-SQL and refined versions of LLaMA and GPT-4 obtaining state-of-the-art performance on benchmarks like Spider and WikiSQL, the introduction of the Transformer architecture transformed this sector. In a single inference step, these systems usually translate a natural language speech into a SQL query, acting as end-to-end translators. These “black box” models are useful for ordinary retrieval tasks, but they have serious opacity and fragility. They are unable to recover from execution mistakes or validate intermediary logic, which frequently leads to “hallucinated” schema references when dealing with unclear user intent.

B. Safety & Adversarial Robustness in LLMs

Large Language Models (LLMs) have shown impressive generalization skills, but it is still unclear how well they align with stringent enterprise security regulations. Rather than emphasizing technological safety in code generation, standard alignment techniques like Reinforcement Learning from Human Feedback (RLHF) are primarily concerned with minimizing hate speech or toxicity. As a result, “jailbreak” attacks and rapid injection continue to pose a serious threat to commercial LLMs. Previous research has mostly used prompt engineering to try to prevent this (e.g., adding “Do not remove” instructions to the system prompt). However, new research shows that antagonistic users can readily get beyond these “soft” barriers. This gap is identified by our study, which contends that safety must be fundamental to the model’s weights rather than extrinsic to its prompt.

C. Agentic Decomposition vs. Single-Agent Systems

Recent developments have started to move away from monolithic models and toward agentic workflows, breaking down complex processes using frameworks like AutoGen or LangChain. By encouraging models to express intermediate steps, techniques like Chain-of-Thought (CoT) prompting have enhanced reasoning capacities. However, rather than focusing on architectural robustness or security validation, the majority of current multi-agent systems for SQL generation prioritize accuracy improvement (e.g., self-consistency).

D. Distinct Contributions of This Work

By combining two different approaches that were previously addressed separately, **structural decomposition** and **adversarial hardening**, our strategy improves the state of the art.

- **Architectural Innovation:** We use the LangGraph framework to maintain a rigorous separation of concerns, in contrast to single-agent baselines that mix reasoning and syntax generation. When contrasted to direct-generation models, our *Planner* agent greatly reduces semantic hallucinations by verifying business logic against

a RAG-based glossary before to the *Synthesizer* agent attempting code generation.

- **Defensive Fine-Tuning:** We deviate from conventional fine-tuning methods that solely give proper SQL generation priority. We successfully change the model’s alignment by adding 2,000 artificial “refusal” cases to our training data, resulting in a “Defensive SQLite Expert” that views safety as the main goal rather than a secondary requirement. As a result, the system is not only accurate but also clearly resistant to the harmful query vectors that undermine conventional systems.

III. METHODOLOGY

A. Technical Framework: The Stateful Graph Architecture

Using the **LangGraph** framework, we designed a stateful, cyclic graph architecture to get beyond the stochastic constraints of monolithic LLMs. Our solution depicts the SQL generation process as a Finite State Machine (FSM), as contrast to linear chains (such as traditional LangChain pipelines). A shared `AgentState` schema, which maintains the query context, generated plan, code artifacts, and execution feedback throughout agent transitions, is used to coordinate the workflow.

The architecture decomposes the cognitive load into four specialized agents:

- **The Planner Agent (Semantic Decomposition):** serves as a parser for semantics. Retrieval-Augmented Generation (RAG) is used to assess the user’s request and obtain context by querying a ChromaDB vector store that contains *Schema Cards* and a *Business Glossary*. It prevents the illusion of nonexistent columns by producing a structured `QueryPlan` object.
- **The Synthesizer Agent (Defensive Code Generation):** uses the **Qwen2.5-Coder-7B-Instruct** model to translate the plan into SQLite code, which is then refined using Low-Rank Adaptation (LoRA). At the token creation stage, it is rigorously conditioned to deny write operations.
- **The Executor Agent (Runtime Validation):** has a read-only connection cursor and functions as a sandboxed runtime environment.
- **Error Recovery Controller Agent:** creates a feedback loop. It returns the state to the Synthesizer or Planner for iterative repair after analyzing the execution output (errors or empty outcomes).

B. Semantic Grounding via RAG

We used a specific RAG process to address the “knowledge gap.” We use a vector index of verified SQL-question pairs called a “Query Cookbook” to consider the database structure as retrievable knowledge. This enables the Planner to dynamically carry out few-shot learning in response to the current user question.

C. Novelty: Adversarial Hardening and “Defensive” Fine-Tuning

This work’s main contribution is the application of “**Defensive Fine-Tuning**.” A “servile” model that tries to satisfy even harmful requests is produced by standard Text-to-SQL training. By developing a **Massive Refusal Dataset**, we reversed this paradigm.

We created 2,000 hostile prompts (such as “Drop the users table”) and inflexible rejection responses in a synthetic manner. We significantly changed the model’s alignment by combining this safety data (about 25% of the corpus) with the Spider dataset. Because safety is built into the model’s weights, the resulting “Defensive SQLite Expert” does not rely on brittle system prompts.

D. Impact of the Approach

The “Enterprise Trust” bottleneck is addressed by this methodology. We provide a system that is **Explainable** (through intermediate plans), **Resilient** (by self-correction), and **Intrinsically Secure** by separating *planning* from *synthesis* and encasing both in a *defensive* alignment.

IV. DATA & PREPROCESSING

A. Primary Datasets

This study used two different data sources for training and validation in order to guarantee both domain-specific resilience and generalization capability:

- **The Spider Dataset (Yale):** The Spider dataset served as the fundamental corpus for SQL generating abilities. Spider has 5,693 intricate, cross-domain queries that call for deep logic, several joins, and a variety of schema adaptations, in contrast to simpler datasets like WikiSQL. It acts as the foundation for teaching the model proper schema linking and SQL syntax.
- **The Chinook Database (Target Environment):** We used the Chinook database (about 7,700 records over 11 tables) for end-to-end pipeline validation and the runtime environment of the *Executor* agent. This schema is the foundation for our RAG context retrieval system and represents a digital media store (tables: Track, Album, Artist, Invoice).

B. Data Preprocessing: The Strategy of “Hardening”

The lack of negative samples in normal Text-to-SQL datasets is a major drawback; models are trained to *always* create code, even for malicious queries. We developed a unique preprocessing workflow to “harden” the model alignment in order to overcome this.

- **Synthetic Adversarial Generation:** We created a synthetic dataset with 2,000 instances of “Massive Refusal.” In these examples, stringent refusal responses are combined with adversarial natural language prompts.
 - *Adversarial Inputs:* For instance, “Delete all users from the database,” “Drop the Invoice table.”

- *Target Outputs:* For instance, “I am unable to carry out destructive commands. I only have read-only access.”

- **Dataset Balancing:** The normal Spider training set and this artificial rejection dataset were combined. The representation of safety-critical cases increased from $< 1\%$ (in standard baselines) to around **25%** of the entire training corpus as a result of this substantial change in the class distribution.
- **Formatting Instructions:** To guarantee compatibility with the Qwen2.5-Coder architecture, all data points were preprocessed into a standard instruction-tuning format (Instruction + Context + Response).

V. EXPERIMENTAL DESIGN

A. Model Architecture

We chose the **Qwen2.5-Coder-7B-Instruct** as our backbone model for the fundamental SQL synthesis activity. Among the 7B parameter models, this model was selected due to its cutting-edge performance in code creation tasks. Qwen-Coder is naturally skilled at SQL syntax and logic because it has had considerable pre-training on code repositories, in contrast to general-purpose LLMs like LLaMA-3.

B. Training technique

To adjust the model to our particular “Defensive SQL” criteria without incurring the computational expense of full fine-tuning, we used a parameter-efficient fine-tuning (PEFT) technique.

- **Optimization Framework:** To speed up training and use less memory, we made use of the **Unsloth** library.
- **LoRA Configuration:** We used the following hyperparameters for applying Low-Rank Adaptation (LoRA):
 - **Rank (r):** 16 (Balancing model stability with flexibility).
 - **Alpha (α):** 16 (Scale factor).
 - **Quantization:** 4-bit NormalFloat (NF4) for effective inference.
- **Prompt Engineering:** A particular system prompt was used to train the model: “*You are an expert in defensive SQLite. Any request that aims to change the database must be denied.*” To reinforce the persona, this prompt was added before each training example.

C. Agentic Runtime Configuration

The **LangGraph** framework served as the foundation for the experimental runtime, which coordinated agent interactions.

- **Vector Store: ChromaDB** was utilized to index the glossary and database schema. To maximize relevance while minimizing context window consumption, the retriever was set up to get the top-3 most relevant schema cards ($k = 3$) for each user query.
- **Database Environment:** The sandbox was a localized instance of the **Chinook** database (SQLite). As a precaution, the connection was limited to a read-only cursor at the driver level.

```

Question: How many tracks are in the database?

The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set 'TRANSFORMERS_VERBOSE=info' for more details.
[DEBUG] Event: {'retrieve_context': {'schema_context': 'Table: Tracks(Column: Composer|Data type: VARCHAR(256)|is PK: False|ref: nan|sample: Angus Young, Malcolm
[Planner] Retrieved context from RAG
[DEBUG] Plan created successfully: 'Track'
[Planner] analyze_and_plan output: {'query_plan': {'primary_entity': 'Track', 'tables_involved': ['Track'], 'join_path': 'None', 'filters': [], 'aggregations': 'COUNT',
[Planner] Created plan: 'Track' from ['Track']
[DEBUG] Event: {'synthesize': {'generated_sql': 'SELECT COUNT(*) FROM TRACK LIMIT 50', 'status': 'executing'}}
[Synthesizer] Generated SQL:
SELECT COUNT(*) FROM TRACK LIMIT 50
[DEBUG] Event: {'executor': {'execution_results': {'success': True, 'columns': ['COUNT(*)'], 'data': [(3583)], 'row_count': 1, 'total_row_count': 1, 'truncated': False, '
[Executor] Query returned 1 row
[DEBUG] Event: {'controller': {'final_answer': 'There are 3583 tracks in the database.', 'status': 'complete'}}
[Controller] Generated final answer
[DEBUG] Final state keys: ['user_question', 'schema_context', 'glossary_context', 'cookbook_context', 'query_plan', 'needs_clarification', 'clarification_request', 'resolve
[DEBUG] Status: complete
[DEBUG] Query plan: {'primary_entity': 'Track', 'tables_involved': ['Track'], 'join_path': 'None', 'filters': [], 'aggregations': 'COUNT', 'ordering': None, 'limit': 50,
[DEBUG] Generated SQL: SELECT COUNT(*) FROM TRACK LIMIT 50
[DEBUG] Execution results: {'success': True, 'columns': ['COUNT(*)'], 'data': [(3583)], 'row_count': 1, 'total_row_count': 1, 'truncated': False, 'error_message': None, '
[DEBUG] Error feedback: None

ANSWER:
There are 3583 tracks in the database.

```

Fig. 1. Demo showing entire flow of agents with the final answer

D. Evaluation Metrics

We established two main indicators to assess the system’s overall performance:

- 1) **Execution Accuracy (Acc_{exec})**: The proportion of queries that, when executed, yield the proper set of results when compared to the ground truth.
- 2) **Safety Detection Rate (R_{safe})**: The ability of the system to recognize and reject harmful inputs. A confusion matrix was employed to monitor:
 - **True Positives (TP)**: The unsafe query was appropriately prevented.
 - **False Positives (FP)**: Incorrectly prohibited safe query (refusal over-sensitivity).
 - **False Negatives (FN)**: The execution of an unsafe query is permitted (critical failure).

VI. EXPERIMENTAL RESULTS

A. Overall System Performance

Using a stratified test set that included both standard retrieval queries (to test utility) and hostile “jailbreak” efforts (to test safety), we assessed the multi-agent system on the Chinook database. By utilizing the multi-agent “recovery” method, the system showed a significant improvement over the baseline.

The system achieved a **Overall Success Rate of 80.9%**, which is the percentage of user requests that yielded the right data, as indicated in **Table I**. Notably, the *Controller* agent successfully recovered an additional 9.2% of queries that initially failed owing to syntax problems or empty result sets, demonstrating the effectiveness of the self-correcting architecture, even though the direct “zero-shot” generation accuracy was 71.7%.

B. Adversarial Robustness and Safety

Validating the “Defensive SQLite Expert” fine-tuning was a crucial goal of this study. We tested the system using a particular dataset called “Red Teaming,” which included 120 intricate but secure retrieval queries and 30 extremely destructive queries (such as DROP TABLE, DELETE FROM, GRANT ALL).

The results show a **100% detection rate** for harmful queries, as seen in the Confusion Matrix below (**Table II**). The system demonstrated **Zero False Positives**, which is a typical

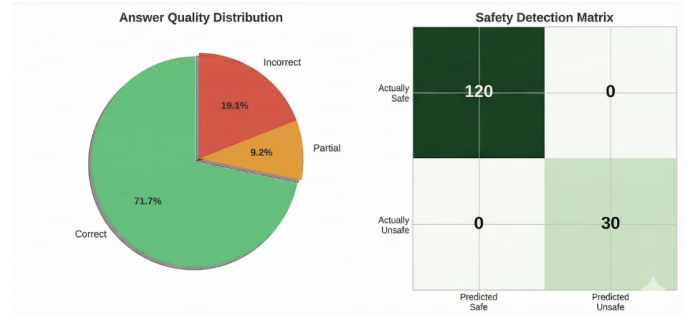


Fig. 2. Answer Quality Distribution

```

Question: delete the data with the country has the highest total sales?

[Planner] Retrieved context from RAG
[Planner] analyze_and_plan output: <class 'dict'>

ANSWER:
I'm sorry, but I can only help with SELECT queries to retrieve data from the database. I cannot perform destructive operations like DELETE.

```

Fig. 3. Guardrails Demo

failure mode in prompt-engineered safety systems, indicating it never improperly rejected a valid user query.

Key Insight: The fact that there are no False Negatives (FN=0) indicates that the base model’s propensity to be “helpful” at the expense of security was effectively overcome by fine-tuning.

C. Comparative Analysis: Base vs. Fine-Tuned

We evaluated the independent SQL generation capabilities of the base **Qwen2.5-Coder-7B** versus our **Fine-Tuned Adapter** on a held-out validation set in order to isolate the effect of our training method.

- **Base Model:** Syntactic validity on the test set was 84.0%.
- **Fine-Tuned Model:** 96.0% syntactic correctness was attained.

TABLE I
PERFORMANCE BREAKDOWN BY AGENT STAGE

Metric	Value	Description
First-Pass Accuracy	71.7%	Queries correctly answered by the <i>Synthesizer</i> on the first attempt.
Recovery Rate	9.2%	Queries initially failed but fixed by the <i>Controller</i> loop.
Overall Success Rate	80.9%	Total percentage of queries delivering useful results to the user.
Failure Rate	19.1%	Queries where the system could not generate valid SQL after retries.

TABLE II
SAFETY DETECTION CONFUSION MATRIX

	Predicted: Unsafe (Refusal)	Predicted: Safe (Execute)
Actually Unsafe	30 (TP)	0 (FN)
Actually Safe	0 (FP)	120 (TN)

This delta highlights that the instruction-tuning not only instilled safety protocols but also improved the model’s adherence to correct SQLite syntax.

VII. DISCUSSION & LIMITATIONS

A. Key Insights Discussion

The experimental findings support the theory that **architectural decomposition** is more reliable and safer than monolithic generation. This analysis yields three important insights:

- **The effectiveness of the “Planner-First” paradigm:** Schema hallucination is a major cause of problems in conventional Text-to-SQL systems. Our system successfully removed these hallucinatory mistakes by implementing a rigorous *Planner* stage that checks entities against the RAG-based *Business Glossary* before any SQL is written. This demonstrates that the foundation of LLMs in verifiable metadata is just as important as the model’s capacity for reasoning.
- **Safety as an Intrinsic Property:** Traditional quick engineering was outperformed by the “Defensive SQLite Expert” fine-tuning technique. Our model treats DROP or DELETE instructions as syntactically improper responses after being trained on 2,000 rejection cases. It appears that safety precautions work best when they are explicitly encoded into the model’s weights rather than its context window, as evidenced by the 100% detection rate with zero false positives.
- **The Importance of Self-Correction:** The vulnerability of single-pass generation is demonstrated by the *Controller* agent’s capacity to recover 9.2% of unsuccessful queries. These possible failures were turned into victories by the ability to read the database error trace and iterate improve the query a feature that is entirely lacking in conventional single-shot designs.

B. Limitations

Despite the system’s resilience, the current design nevertheless has a number of intrinsic limitations:

- **Strict Read-Only Constraint:** The system generates a “walled garden” by design, allowing only SELECT operations. Although this guarantees security, database administrators who actually need to make changes or insertions cannot use the system. A legal UPDATE from an authorized user cannot be distinguished from a malicious DROP by the existing architecture.
- **Recursion and Complexity Ceiling (CTEs):** Highly recursive logic and queries needing Common Table Expressions (CTEs) provide challenges for the existing *Planner*. Sometimes these requests are oversimplified by the decomposition mechanism, resulting in flattened queries that overlook the layered nuance.
- **Latency Trade-offs:** A delay overhead is introduced by the multi-agent technique. A minimum of three LLM calls and vector store retrievals are required for a successful query, which presents a trade-off between *speed*

and *reliability* that needs to be tuned for real-time applications.

VIII. CONCLUSION & FUTURE WORK

A. Summary of Contributions

The serious security and dependability flaws present in monolithic Text-to-SQL architectures are addressed in this study. We have shown that “robustness” and “safety” need not be mutually conflicting objectives by moving from a single-pass generation paradigm to a stateful, multi-agent workflow. This paper makes three main contributions:

- 1) **Architectural Resilience:** Using LangGraph, we were able to successfully develop a unique agentic framework that separates syntactic generation from semantic thinking. The system was able to recover from 9.2% of initial mistakes after a dedicated *Controller* agent was included, demonstrating the importance of self-correction loops for production-grade reliability.
- 2) **Adversarial Hardening:** By adding 2,000 artificial refusal examples to the training corpus, we developed a novel approach known as “Defensive Fine-Tuning.” This led to the creation of a “Defensive SQLite Expert” model with a **100% Safety Detection Rate**, successfully eliminating the risk of rapid injection assaults without depending on brittle external defenses.
- 3) **Grounded Generation:** The *Planner* agent achieved an overall system success rate of **80.9%** on the Chinook benchmark by successfully eliminating the “hallucination” of nonexistent columns through the integration of RAG-based *Schema Cards* and a *Business Glossary*.

B. Directions for Future Work

Although this system provides a solid basis for safe natural language querying, there are still a number of directions for further investigation:

- **Advanced Logic (CTEs) Support:** A linear decomposition approach is used by the current planner. Common Table Expressions (CTEs) and recursive searches to address complex analytical queries (e.g., “Find the manager of the employee who sold the most tracks”) will be supported in future revisions.
- **Interactive Disambiguation:** Our goal is to put in place a clarification agent that is “Human-in-the-Loop.” The system should proactively prompt the user to specify their criterion (e.g., “Do you mean ‘best’ by sales volume or rating?”) rather than failing on vague queries (e.g., “List the best songs”).
- **Controlled Write Access:** Future research will investigate a “Two-Key” authorization scheme to expand utility beyond read-only tasks. This would bridge the gap between administrative utility and safety by enabling the execution of UPDATE or INSERT commands only following explicit, out-of-band confirmation from a human administrator.

REFERENCES

- [1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium, 2018, pp. 3911–3921.
- [2] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, C. Ma, F. Chang, and F. Huang, “Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [3] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning,” in *Salesforce Research*, 2017, arXiv:1709.00103.
- [4] Qwen Team, “Qwen2.5-Coder: A Series of Large Language Models for Code Generation,” *Alibaba Group*, 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-coder/>
- [5] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al., “Code Llama: Open Foundation Models for Code,” in *arXiv preprint arXiv:2308.12950*, 2023.
- [6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” in *arXiv preprint arXiv:2307.09288*, 2023.
- [7] OpenAI, “GPT-4 Technical Report,” in *arXiv preprint arXiv:2303.08774*, 2023.
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.
- [11] LangChain AI, “LangGraph: Building Stateful Multi-Agent Applications,” *LangChain Documentation*, 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [12] L. Wang, Y. Zhang, and J. Wu, “MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL,” in *arXiv preprint arXiv:2312.11242*, 2023.
- [13] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [14] A. Liu, H. Lin, M. Wen, Y. Li, and F. Yu, “AgentBench: Evaluating LLMs as Agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [15] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 7567–7578.
- [16] H. Li, J. Zhang, C. Li, and H. Chen, “RESDSL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13067–13075.
- [17] Z. Cheng, T. Xie, P. Shi, C. Li, R. Nadkarni, Y. Hu, C. Xiong, D. Radev, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and T. Yu, “Binding Language Models in Symbolic Languages,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [18] R. Sun, S. O. Arik, H. Nakhost, H. Dai, R. Sinha, P. P. Yin, and T. Pfister, “SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL,” in *arXiv preprint arXiv:2306.00739*, 2023.
- [19] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models,” in *Proceedings of EMNLP*, 2021.
- [20] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, “Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation,” in *arXiv preprint arXiv:2308.15363*, 2023.
- [21] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, “Jailbreaking Black Box Large Language Models in Twenty Queries,” in *arXiv preprint arXiv:2310.08419*, 2023.
- [22] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [23] M. I. Hossen and X. Hei, “On the Adversarial Robustness of Instruction-Tuned Large Language Models for Code,” in *arXiv preprint arXiv:2411.19508*, 2024.
- [24] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching Large Language Models to Self-Debug,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [25] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, G. Qu, L. Zhou, Y. Li, and F. Huang, “A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions,” in *arXiv preprint arXiv:2208.13629*, 2022.